

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнила: А. О. Полевая
Группа: М8О-309БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: познакомиться с основами 3D-графики: построением 3D-объектов, их проекцией на плоскость экрана, матрицами перспективной и ортографической проекций и аффинными преобразованиями.

Вариант 6: пирамида.

Программа написана на C++20 с использованием Vulkan, GLFW и Dear ImGui на основе стартового проекта, работает в реальном времени. Выполнены все дополнительные задания:

1. переключение ортографической и перспективной проекций;
2. изменение позиции, поворота и масштаба фигуры;
3. движение и вращение по сложной траектории, пауза, скорость и параметры траектории;
4. выбор цвета фигуры (ColorEdit);
5. процедурные цвета вершин, умножаемые на выбранный цвет;
6. несколько объектов, у каждого свой набор дескрипторов.

Метод решения

Геометрия

Основание пирамиды — квадрат 1×1 в плоскости $y = -0,5$, вершина — точка $(0; 0,5; 0)$. Всего 5 вершин и 6 треугольников (2 в основании и 4 боковых), 18 индексов uint16. Вершины каждого треугольника идут против часовой стрелки при взгляде снаружи, чтобы отсечение задних граней скрывало только невидимые грани.

Аффинные преобразования и камера

Точки записываются в однородных координатах, преобразования — матрицами 4×4 . Матрица модели:

$$M = T(c + p(t)) \cdot R_y(\theta) \cdot R_z(\gamma) \cdot R_y(\beta) \cdot R_x(\alpha) \cdot S(s),$$

где c — позиция из интерфейса, $p(t)$ — точка траектории, α, β, γ — углы поворота, θ — угол собственного вращения, s — масштаб по осям. Матрицы применяются справа налево: масштаб, поворот, вращение, перенос. Масштаб ограничен положительными значениями: отрицательный меняет направление обхода треугольников, и они пропадают при отсечении.

Камера стоит в точке $(0, 0, 8)$ и смотрит в начало координат. Положение вершины на экране: $P \cdot V \cdot M \cdot x$, где V — матрица вида, P — проекция.

Проекции

В Vulkan ось Y пространства отсечения направлена вниз, а глубина лежит в $[0, 1]$, поэтому матрицы проекций записаны с учётом этого.

Перспективная проекция (угол обзора 60° , плоскости 0,1 и 100): $w = -z$, поэтому после деления на w удалённые объекты уменьшаются; ближняя плоскость переходит в глубину 0, дальняя — в 1; минус во второй строке переворачивает ось Y .

Ортографическая проекция линейно переводит параллелепипед $[l, r] \times [b, t] \times [-far, -near]$ в объём отсечения без деления на расстояние: размер не зависит от удалённости, параллельные рёбра остаются параллельными. Высота объёма равна $2d \cdot \tan(fov/2)$, как у перспективной камеры в плоскости $z = 0$, поэтому при переключении размер фигуры в центре не меняется.

Траектория и анимация

$$p(t) = (R \cdot \cos a, H \cdot \sin(k \cdot a), R \cdot \sin a), \quad a = t + \varphi,$$

где R — радиус, H — высота волны, k — целое число волн за оборот, φ — сдвиг объекта (для объекта i : $\varphi = 2\pi \cdot i/8$). В интерфейсе меняются R , H , k , скорость движения, скорость вращения вокруг вертикальной оси и пауза. Параметр накапливается по кадрам: $t \leftarrow t + v \cdot \Delta t$, поэтому пауза и смена скорости не вызывают скачков. При целом k траектория периодична с периодом 2π , и t хранится по модулю 2π , чтобы не терять точность float.

Цвет

Цвет вершины вычисляется по её локальным координатам: $color = position + 0,5$ (значения в $[0, 1]$), между вершинами он интерполируется. Итоговый цвет — произведение цвета вершины на цвет из ColorEdit; опция «Vertex colors» отключает цвета вершин. Так как буфер кадра в формате $*_SRGB$, цвета вершин переводятся из sRGB в линейное пространство (степень 2,2).

Используемые объекты Vulkan

- `VkBuffer` и `VmaAllocation`: вершинный буфер (5 вершин), индексный буфер (18 индексов) и по одному uniform-буферу на объект (матрицы M , V , P , цвет и флаг, std140, 224 байта);
- `VkShaderModule`: вершинный и фрагментный шейдеры (GLSL, SPIR-V через glslc);

- `VkDescriptorSetLayout`, `VkDescriptorPool`, `VkDescriptorSet`: восемь наборов, по одному на объект, каждый указывает на его `uniform`-буфер;
- `VkPipelineLayout` и `VkPipeline`: конвейер с топологией `TRIANGLE_LIST`, отсечением задних граней, тестом глубины и динамическими `viewport` и `scissor`;
- `VkRenderPass` и `VkFramebuffer` из стартового проекта;
- `VkCommandBuffer`: каждый кадр записывает привязку конвейера и буферов, затем для каждого объекта привязку его набора дескрипторов и `vkCmdDrawIndexed`.

Несколько объектов

Команды выполняются на GPU после окончания записи. Если бы все пирамиды брали данные из одного `uniform`-буфера, они бы нарисовались по данным последнего объекта. Поэтому у каждой пирамиды свой `uniform`-буфер и свой набор дескрипторов, который привязывается через `vkCmdBindDescriptorSets` перед каждым `vkCmdDrawIndexed`. Вершинный и индексный буферы общие. Буферы и наборы создаются сразу для восьми объектов, поэтому `Add` и `Remove` ничего не выделяют во время работы.

Результаты

На рисунке 1 — пирамида в перспективной проекции с цветами вершин; она движется по траектории и вращается. Слева панель управления.

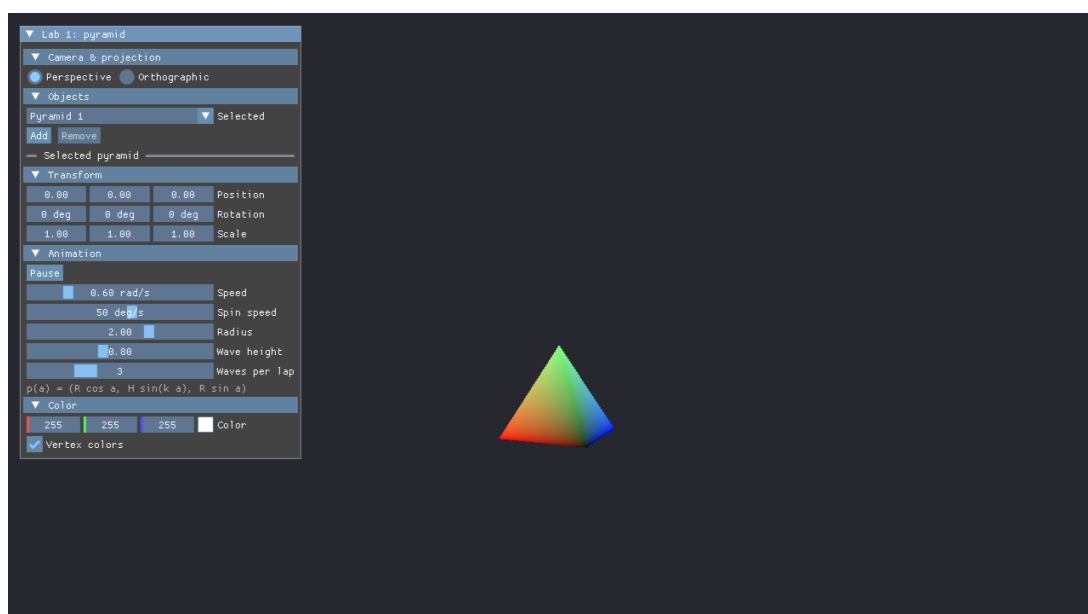


Рисунок 1 — Перспективная проекция

На рисунке 2 — ортографическая проекция, анимация на паузе. Пирамида повернута (Rotation 30° по X, 15° по Y) и растянута (Scale 1,5 по X и 4 по Y). Видны основание и две боковые грани, параллельные рёбра остаются параллельными, перспективного сокращения нет.

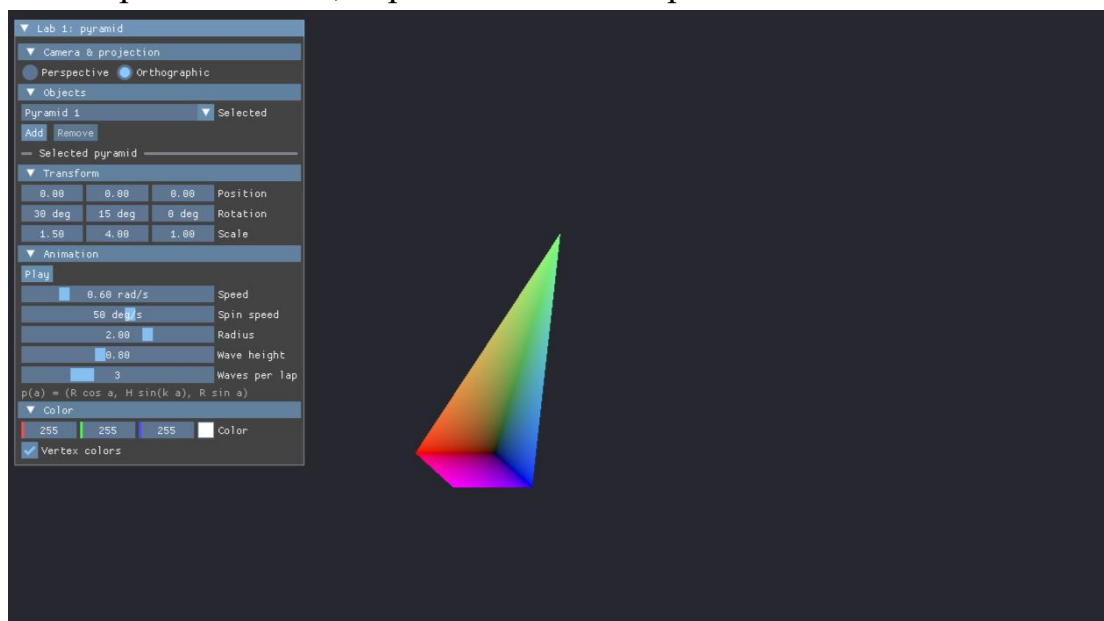


Рисунок 2 — Ортографическая проекция и аффинные преобразования

На рисунке 3 — семь пирамид. Они стартуют из разных точек траектории, а параметры траектории и цвет у каждой свои: у выбранной пирамиды 7 изменены радиус, скорость, высота волны и цвет. Ближние пирамиды закрывают дальние благодаря тесту глубины.

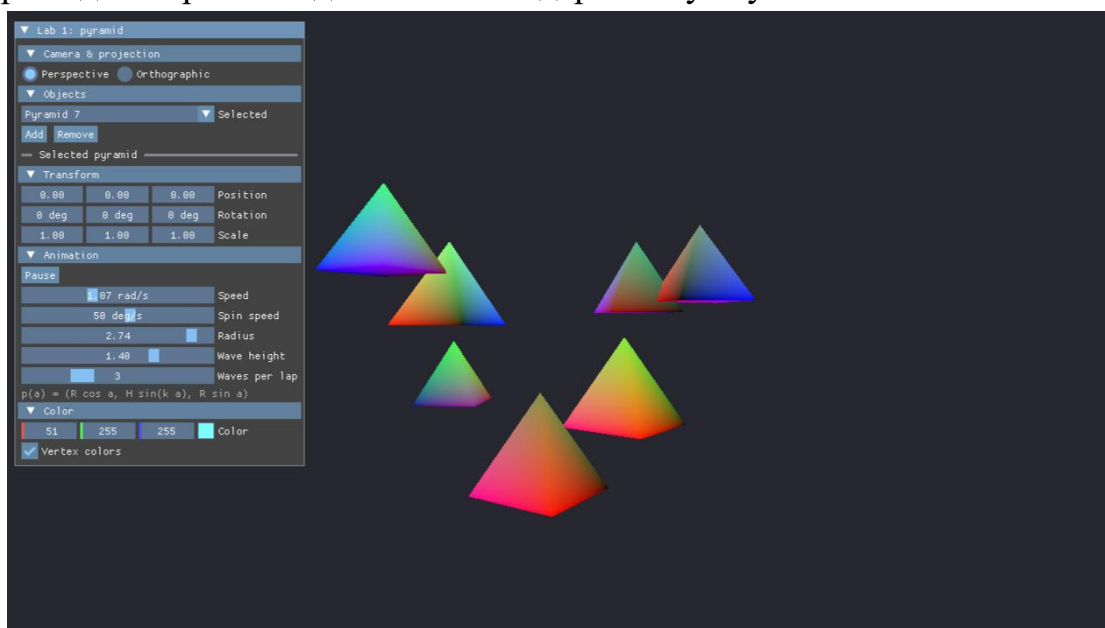


Рисунок 3 — Несколько объектов с отдельными параметрами

Выводы

Я научилась строить 3D-объект и отрисовывать его в реальном времени с помощью Vulkan, применять матрицы переноса, поворота, масштаба, вида и проекций (перспективной и ортографической) с учётом соглашений Vulkan, передавать данные в шейдеры через uniform-буферы и наборы дескрипторов и рисовать несколько объектов с отдельным набором дескрипторов на каждый.